



南京大學

本科畢業論文

学 院	智能软件与工程学院
专 业	软件工程（智能化软件）
题 目	实践中的理论最优哈希表
年 级	2022 学 号 221900323
学生姓名	纪泽操
指导教师	刘明谋 职 称 副教授
提交日期	2026 年 5 月 30 日



南京大学本科毕业论文（设计） 诚信承诺书

本人郑重承诺：所呈交的毕业论文（设计）（题目：实践中的理论最优哈希表）是在指导教师的指导下严格按照学校和学院有关规定由本人独立完成的。本毕业论文（设计）中引用他人观点及参考资源的内容均已标注引用，如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文（设计）等违纪行为。

作者签名：纪泽操

学号：221900323

日期：2026 年 5 月 30 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：实践中的理论最优哈希表

学院：智能软件与工程学院

专业：软件工程（智能化软件）

本科生姓名：纪泽操

指导教师（姓名、职称）：刘明谋 副教授

摘要：

本论文以《On the Optimal Time/Space Trade off for Hash Tables》为研究方向，系统整理文中提及的理论算法及核心思想，然后加以简化并落实到实际当中，从宏观架构层面，搭建了 Facility - Cubby - Slot 三层存储体系，从而达成高效地组织和运作哈希表的目的；微观机制层面，融合了可逆哈希函数，局部查询路径，商压缩，K - Kick 规则，多层级 Cubby 以及双表渐进式扩缩容迁移等主要技术，虽然有些执行过程采取了简化办法，没有完全按照最初理论模型应有的严谨程度来操作，但是整个哈希系统的功能还是齐全的，逻辑上也是连贯的，可以稳定运行。落地的哈希系统包含初始化，插入，删除，查询，扩容以及性能指标验证等完整的操作，实验结果表明，该系统可以稳定执行各类动态操作，其路由访问步数，元素被踢出的移动次数以及元数据编码位数等关键指标均处于可控制范围内，能够精准再现原始理论架构的核心逻辑，而且，本文明确了工程简化达成与严格理论模型之间的差异界限，给类似紧凑哈希结构的工程化重现及其在实际场景中的应用赋予了有价值的参考。

关键词：哈希表；时间空间权衡；探测复杂度；增强开放寻址；k-kick tree；动态扩容

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: On the Optimal Time/Space Tradeoff for Hash Tables

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

This thesis is built on the study of the paper *On the Optimal Time/Space Tradeoff for Hash Tables*. It does an analysis on the theoretical algorithms and main ideas in the paper, and then tries to make them work in real engineering by using a more simple and practical way. From the overall structure point of view, a three-layer storage model made up of Facility, Cubby, and Slot is designed, which helps to manage the hash table in a more organized way. At the implementation side, the system uses many key techniques like reversible hash functions, localized query paths, quotient compression, the K-Kick rule, multi-tier Cubby organization, and incremental dual-table resizing and migration. Even though some parts are simplified and do not fully follow the strict assumptions in the original theory, the system still keeps the main framework and runs with stable logic. The built hash system can support all basic functions such as initialization, insertion, deletion, lookup, dynamic resizing, and performance testing. The experiment results show that it can handle different dynamic operations stably, and important indicators like routing probe counts, kick displacement frequency, and metadata encoding overhead are kept in acceptable levels. The implementation is able to get the core behavior of the original theoretical design in a reasonable way. Also, this work makes a discussion about the differences between the engineering simplifications used here and the pure theoretical model, which can be helpful for the practical rebuilding and use of compact hash table structures.

KEYWORDS: hash tables; time–space tradeoff; the Probe-Complexity problem; enhanced open addressing; k-kick tree; dynamic expansion

目 录

目 录	III
第一章 引言	1
1.1 实验背景及意义	1
1.2 国内外现状	2
1.3 实验路线	3
第二章 问题模型与理论基础	4
2.1 探测复杂度与球槽分配模型	4
2.2 k-kick 树分层驱逐思想	4
2.3 紧凑元数据存储核心思想	5
2.4 分层架构整体设计思想	6
2.5 商压缩的信息论最优思想	6
2.6 算法层次全景总结	7
第三章 系统设计与实现	8
3.1 总体架构与数据路径	8
3.2 参数产生与地址拆分	9
3.3 Facility、Cubby 与空闲槽结构	10
3.3.1 tail 管理	11
3.3.2 j-tiered cubbies 数量与重建调度	11
3.4 MiniArray 与 MetaEntry	13
3.4.1 MiniArray 变长元数据存储结构	13
3.4.2 MetaEntry 元数据结构	14
3.5 Router 组件	15

3.6	商压缩与键恢复	16
3.7	双表渐进迁移	17
3.8	插入路径与 k-kick 实现	19
3.9	查询与删除路径	20
第四章	系统验证与性能实验	22
4.1	实验目的	22
4.2	性能实验设置	22
4.3	宏基准结果	22
4.4	结构性指标分析	23
4.5	小结	24
第五章	总结与展望	25
5.1	工作总结	25
5.2	不足与局限	25
5.3	可改进方向	26
参考文献		27
致 谢		29

第一章 引言

1.1 实验背景及意义

哈希表是一种通过哈希函数把关键字映射到数组相应位置的结构^[1]，它在数据库索引，键值存储，编译器符号表以及运行时系统等常见基础架构中被广泛应用。相较于链地址法，开放寻址哈希表将元素直接放置在连续数组中，其查询路径相对较短，缓存局部性表现更优；但是当负载因子上升时，冲突及探测长度会快速影响操作成本。在实际工程操作中，常采用预留空槽的方式以降低冲突发生几率，但是这也会致使单位元素所占据的附加空间有所增加。因此哈希表的设计需解决如下具体问题：在确保查询插入及删除操作时间可有效控制的基础上，如何减少每个元素为实现定位，更新与恢复而额外存储的信息量。

论文《On the Optimal Time/Space Tradeoff for Hash Tables》从理论层面阐述了前面提到的问题^[2]，这篇论文提到，在只能花费常数时间执行查询的前提下，如果插入和删除要承担 $O(k)$ 个元素的交换代价¹时，每个元素所占的额外空间就能从 $O(\log \log n)$ 比特缩减到 $O(\log^{(k)} n)$ 比特²³，这个结果说明，开放寻址哈希表的空间利用率并非仅仅取决于空位占比，而且和元素转移，局部引导以及键重建信息的编码方法存在紧密联系。此前，紧凑哈希表在常数时间操作下每键约需 $O(\log \log n)$ 比特冗余，动态检索与 filter 的下界研究^[3] 及 Iceberg 哈希^[4] 等结果已逐步逼近信息论下界。凭借此理论，本文针对 Facility⁴、Cubby⁵、Router⁶、

1 一次更新过程中被搬移（踢出或回填）的元素个数；允许有界交换可换取更低的附加空间。

2 本文探讨的空间开销都是按照比特计算的。

3 符号 $\log^{(k)} n$ 就是连续对 n 取 k 次对数，也就是 $\log \log \dots \log n$ （一共 k 个 $\log$ ）。

4 将整个哈希表看作一个数组，划分为均等的管理单元，命名为 Facility。

5 Facility 内继续划分的局部存储容器，包含槽位数组、变长元数据及空闲槽索引结构，是元素放置与局部驱逐的基本单位。

6 局部查询路由器（Local Query Router）：在单个 Facility 的某个桶内，将键映射到候选（Cubby, slot(槽位)）的紧凑前缀树结构。

MiniArray¹、商压缩²以及双表渐进迁移³等主要架构展开简化重现并加以实现，再经由测试考察这些架构在关键操作流程中的真实运行状况。

1.2 国内外现状

哈希表实现主要包括经典的链地址法和开放寻址法。链地址法借助链表或者桶结构来处理冲突，以达成直接的效果；但是指针，动态分配以及额外的桶结构会引发空间和缓存方面的开销。开放寻址法将元素放置于数组槽位之中，其访问局部性较为突出；但是性能会受到负载因子，探针序列以及删除策略的较大影响^[1]。在此基础上，布谷鸟哈希通过多个候选位置和踢出过程缩短查询路径^[5]，完美哈希面向静态集合提供更强的查找保证^[6]，其动态情形亦有大量研究^[3]；IcebergHT 等工程实现^[7] 则进一步在稳定性与低关联度下优化实际吞吐。这些结构分别对查询的确定性，构造途径或空间的利用效率起到改善作用，但是在动态进行插入，删除操作与拥有较低附加空间这两者之间，仍然存在难以同时满足的限制条件。

近年来理论性的探讨进一步将聚焦点推进到关于位级空间开销的层面上。仅仅依靠负载因子衡量空间利用率，无法充分解释紧凑哈希表⁴的成本，原因是系统还需留存用于支持查询，更新以及键恢复的辅助信息。在这一视角下所提出的正是本文所参考的最优哈希表理论⁵，该理论把元素的放置看作球槽分配问题^[8-9]，并且借助 k-kick 树⁶、Local Query Router、MiniArray 以及商压缩来组合出新型的时间-空间权衡方式。本文的工作范畴限定于工程方面的实现以及实验层面的观察，即围绕这些可实现的结构开展简化的复现工作，简化的理论机制，以及此类简化对实验结果解读产生的影响。

-
- 1 Cubby 内按槽位存放变长比特元数据的打包数组，用于保存恢复原始键值所需的差分等信息。
 - 2 用哈希映射函数 $y = \pi(x)$ ，将原始键值映射到 y ， y 的低位用于寻址后，槽位仅保存剩余高位商（payload），从而降低每元素存储位数。
 - 3 扩缩容时不一次性重哈希全表，而是维护 active 与 old 两张表，在后续操作中按预算逐步将 old 中元素迁入 active。
 - 4 在尽量降低每元素附加比特数的前提下，仍支持动态插入、删除与常数时间查询的哈希表设计。
 - 5 指 Bender 等人在 STOC 2022 提出的最优时间/空间权衡哈希表理论框架^[2]，在允许有界交换的前提下刻画查询时间与附加空间之间的最优折中。
 - 6 将 Cubby 内槽位按层级抽象为 $k + 1$ 层区间结构（仍以数组实现），深层区间对应更高放置优先级；插入时沿深层向低层方向进行有界踢出。

1.3 实验路线

本文以《On the Optimal Time/Space Tradeoff for Hash Tables》中的关键构造为核心，进行简化的复刻以及实际的应用推广。系统借助 Facility-Cubby-Slot 来组织局部存储，每个 Facility 维护一组 Router，依靠这些 Router 将键精准定位到相应的 Cubby 和槽位；槽位中存储商压缩后的 payload¹，而 MiniArray 保存用于恢复 $\pi(x)$ ²所需的变长元数据；进行扩缩容操作时，采用 active/old 双表渐进迁移的方式。将重点置于插入，查询，删除，键的恢复以及迁移路径方面，同时记录踢出次数、Router 的访问步数，元数据的位数等相关指标。

本文针对三个相互联系的问题依次展开论述：

1. 当下的完成情况怎样，能否大批量执行数据插入，精确地实施目标查询，执行删除操作。
2. 在实际运行过程中，插入移动的次数、Router 访问的步数以及逻辑元数据的位数各有怎样的特性。
3. 已有的成果同论文的框架相比作了哪些精简，这些精简又会给实验结论的应用范围带来什么样的局限。

后续的章节里，第二章论述论文《On the Optimal Time/Space Tradeoff for Hash Tables》相关的理论根基，第三章阐述系统的设计与实现，第四章报告功能检测及微基准的结果，第五章归纳已开展的工作，并根据实现的哈希系统和论文所达成的目标之间的差距当作参照去探究后续值得改进的方向。

1 本文中的 payload 指槽位中保存的压缩后有效载荷，主要对应由 $\pi(x)$ 拆出的高位商；它不等同于原始键，必须结合 MiniArray 中的元数据才能恢复用于校验的置换值。

2 键 x 经可逆双射哈希得到的值 g_x ，低比特参与寻址，高位经商压缩存入槽位。

第二章 问题模型与理论基础

2.1 探测复杂度与球槽分配模型

开放寻址哈希表可被抽象为在线状态下的球槽分配问题^[8-9]，假设表里有固定的槽位数，每个槽位最多只能存储一个元素；系统具有动态插入和删除的功能，任何时候元素的数量都不能超过槽位的数量，对于任意键 x ，哈希计算会生成一条预先确定的随机探测序列。

$$h_1(x), h_2(x), \dots,$$

元素只会被安排到此序列指定的诸多候选槽位之中。

当某元素最终被置于其探测序列的第 i 个候选位置时，探测复杂度¹被定义为 $\Theta(1 + \log i)$ 。这一数量可被视为查询期间记录“元素采用第几个候选位置”所需的比特数目，因此与辅助元数据的空间开销存在直接关联。在该模型中有两个指标需同时把控：其一为平均探测复杂度，其对应的是整体附加空间；其二是交换成本，其指的是一次更新过程中被搬移元素的数量。最优哈希表理论所探讨的核心要点，是在允许有限交换的条件下，尽力减小平均探测复杂度。

2.2 k-kick 树分层驱逐思想

k-kick 树并非一种实际的树形数据结构，而是一种规则，此规则将固定长度的数组依照下标抽象成树的形式，第一层包含 $s_0 = n$ 个元素，第二层则包含两个子区间，每个子区间有 $s_1 = \frac{n}{2}$ 个元素，以此类推，下一层的两个元素由上一层均分而来，若已知元素位于某层的区间范围，便能推断出它上层所属的区间，由此生成的树形结构便是 k-kick 树核心理念，但它本质上仍然是数组，这种

¹ 查询时需记录“元素落在第 i 个候选位置”所需的比特数，记为 $\Theta(1 + \log i)$ ，与辅助元数据空间直接相关。

结构会把一个 cubby 中按顺序排列的槽位分配到 $k + 1$ 个逻辑层级之上，而且各层级的大小随层级加深而依次变小，深层的 cubby 更靠近元素的随机偏好位置，因而具有较高的放置优先级；而表层的 cubby 覆盖范围广，主要用于收纳很多无法存放在深层位置的元素。

在进行插入操作之前，哈希过程会为元素明确一条从深层到浅层的 Cubby 路径，并且会给出该元素可使用的最大深度。插入时系统会尝试深层的 Cubby；若目标 Cubby 处于已满状态但仍可调整，则可将其中优先级较低的元素踢出，随后将当前元素放置于该位置，被踢出的元素会再向更浅层寻找合法位置。因为每一次踢出均按照层级方向推进，所以级联进程会受到层数的限制。

参数 k 控制着层数，所以单次更新过程里的迁移次数能被限制在大约 $O(k)$ 这个数量级，论文^[2]表明，在这种交换成本情况下，平均探测复杂度可缩减到大概 $O(\log^{(k)} n)$ ，文章后面会记载 kick_count，而且会在实验说明里把理论 k-kick 树和当下工程版踢出链区分开来。

2.3 紧凑元数据存储核心思想

分层放置虽然会使探测变得不那么复杂，但在执行查询时，仍需知道元素最终处于哪个 Cubby 及其所对应的槽位，理论框架里用到了 MiniArray 和 Local Query Router 这两个部件，MiniArray 用于解决变长元数据的存储问题，Local Query Router 则负责处理局部键与槽位之间的映射关系。

MiniArray 用以存储处于同个 Cubby 之中但长度存在差异的元数据条目，该理论方案借助内存 B 树及查表法来执行变长比特串的打包过程，可以做到在均摊常量时间里完成读写动作，其空间代价与条目的总计比特数成正比关系，不必给每个槽位都按最大条目长度分配空间。

Local Query Router 主要承担对小规模键集合进行本地定位的职责。其将键映射为随机比特串，借助二进制前缀树区分不同的键，之后把树结构以及叶子上的槽位信息编码成连续的比特流。由于每一个 Router 仅负责管理局部的少量元素，因此前缀树的预期规模相对较小，能够在较低的空间开销情况下实现键到 Cubby 以及 Slot 的映射。第三章中的 Router 与 MiniArray 均依照该接口语义来实现，但其更新途径采用了简化处理方式。

2.4 分层架构整体设计思想

本文工作用 Facility - Cubby - Slot 这种三级架构来组织数据，Facility 承担表格区域划分和路由任务，Cubby 负责本地存储及元数据守护，Slot 对应格子中的实际数组位置。

Cubby 是局部存储单元，其内部存有存储槽，可变长度的元数据以及空闲槽检索构造，承担元素的放置，局部的驱逐以及键恢复所涉信息的维护任务。Facility 对多个 Cubby 进行聚合，并且借助局部 Router 将键精准定位到目标 Cubby 以及相应的槽位。理论架构当中还引入了多 tiered Cubby¹以及 tail²机制，非 tail Cubby 尽可能维持处于满载状态，因插入与删除所引发的局部性波动主要由 tail 承受。

为了能够实现动态扩容和缩容，论文提出了多 tiered-cubby 机制，取一个 1-tiered cubbies 作为 tail cubby，每次插入元素都插入 tail cubby 中，为了维护不变量，保持 j -tiered cubby³ 的装满状态以便实现均摊低成本的扩容和缩容以及逼近 1 的装填因子，当删除元素时，删除的元素如果不在 tail cubby 中，就会从 tail cubby 中取出一个元素放入对应的 cubby 中⁴。同时，不同 j -tiered cubbies 的数量已经大小需要维护，设 t_j 为 j -tiered cubbies 的理想数量，如果真实数量比 t_j 低太多，我们会尝试拆分一个 $j + 1$ -tiered cubby 为 t_j 个 j -tiered cubbies，反之过多的 j -tiered cubbies，我们会合成一个 $j + 1$ -tiered cubby，这里不详细实现，具体细节在第三章中详细阐述。

2.5 商压缩的信息论最优思想

正常情况下，哈希表会在槽位里完整保留原始键时，这样就会出现部分信息会和寻址过程产生重复的现象⁵。商压缩借助双射映射⁶，取映射后的低位作为路由，这种情况下，路由已知，我们只需存储剩余的信息和恢复信息，这样就达到

-
- 1 按层级 j 组织不同容量与数量的 Cubby 集合，配合 tail 与拆分/合并维持高装填因子与均摊扩容。
 - 2 各 Facility 唯一允许未满载的 Cubby；新插入默认进入 tail，删除非 tail 元素时由 tail 回填以维持其余 Cubby 接近满载。
 - 3 这里的 tiered cubby 以及不同层次的 j -tiered cubby 会在第三章中详细解释，这里只做简单说明
 - 4 这里放入对应的 cubby 中不是放入刚好空的位置，而是进行插入操作，用于维护对应 cubby 的 k-kick 树的优先级以及不同层次的饱和状态
 - 5 寻址过程会生成一个探测序列，这个序列会告诉我们元素最终处于哪个 Cubby 及其所对应的槽位，而这个探测序列的信息就是部分信息会和寻址过程产生重复的信息。
 - 6 键空间到置换值空间的一一对应映射 π ，保证每个键对应唯一置换值且可逆，从而将寻址信息与存储信息分离。

空间压缩的效果。

具体而言， $\pi(x)$ 的低 $\log N$ 位用来确定 Facility，Router 桶和局部偏好位置；槽位只保存去除该部分之后的 payload，若要复原完整的 $\pi(x)$ ，元数据记载偏好槽位与实际槽位的差值，还有局部桶里的中间位，再加上 Facility 编号，槽位下标和 payload，系统就能重新创建置换值，然后经由 π^{-1} 得到原本的键，第三章会遵照当下代码里的 payload、mid_bits 和 delta 来阐述这个过程。

2.6 算法层次全景总结

上述的理论结构可概括为三方面：

- Facility 负责组织 Cubby、维护 tail、承载扩缩容和重建过程中的局部管理。
- Router 负责把键映射到目标 Cubby 与 Slot，并为查询提供局部定位信息；
- Cubby 负责元素放置、空闲槽检索、局部 k-kick 树和变长元数据编码；

后续的章节会把这三方面当作系统开发的结构参考，并且阐述理论结构和实际达成之间的差别：第二章说明理论机制，第三章对应到代码实现，第四章再依据实际测试结果解释当前实现的行为边界。

第三章 系统设计与实现

本章围绕系统的实际实现细节展开，与第二章的理论描述相比，讨论的是当前实现真实采用的状态组织、操作路径与落地的替代方式。

3.1 总体架构与数据路径

系统所创建的哈希表从逻辑上看是长度为 N 的开放寻址表，而且 N 永远是 2 的幂次，其主要包含五种操作，分别是初始化，插入，查询，删除以及渐进迁移。在初始化阶段要生成 N 和 K ，并形成 Facility 结构；插入阶段首先由 Router 减小候选范围，然后在 tail Cubby 中执行有界 kick；查询阶段借助“Router 候选 + 商压缩恢复校验”来执行命中判断；删除阶段则要释放槽位并执行必要的回填，这些操作过程会记录 `router_probe_steps`¹、`kick_count`²和 `cubby_tier`³，从而可以在后续的实验统计使用。

每张表用 `TableState`⁴来表示，它包含表规模 N ，局部桶数 K 以及 Facility 数组，其中 Facility 负责管理多 tiered cubbies，一个唯一的 tail Cubby 和长度为 K 的 Router 数组，而 Cubby 则是实际的存储容器，Slot 是 Cubby 内的数组槽位。

查询的主路径大致如下，如算法 1 所示：首先计算 $g_x = \pi(x)$ ，按照 g_x 的低 $\log N$ 位找到 Facility 和 Router；Router 给出候选 $(Cubby^*, slot)$ 之后，接着读取槽内的 payload 和 MiniArray 元数据，重新创建 $\pi(x')$ ，并与查询键的置换值做对比。

1 单次操作中 Router 前缀树遍历的步数。

2 单次插入或删除中 k-kick 链发生的槽位搬移次数。

3 本次操作所涉 Cubby 的 tiered 层级编号。

4 哈希表运行时状态，保存元素个数、表规模 N 、Router 桶数 K 及 Facility 数组等。

算法 1 查询过程

- 1: 输入查询键 x
 - 2: 计算 $g_x \leftarrow \pi(x)$
 - 3: g_x 低位拆分出 Facility 编号 r 和 Router 桶号 b
 - 4: 在表中 `facilities[r].D[b]` 调用 `locate(x)`
 - 5: **if** Router 返回候选位置 (C, i) **then**
 - 6: 读取 `C.slots[i]` 和 `C.meta[i]`
 - 7: 重建候选槽位对应的 g'_x
 - 8: 若 $g'_x = g_x$, 返回命中
 - 9: **end if**
 - 10: 返回未命中
-

3.2 参数产生与地址拆分

参数包括 n 、k-kick 参数 k 、负载因子 `load_factor` 和 Feistel 哈希种子¹。初始化时, 代码首先将表规模设置为

$$N = \text{next_pow2}(\max\{2, n\})^2,$$

因此 N 始终为 2 的幂, 且当 $n > 1$ 时满足 $N/2 < n \leq N$ 。随后 `choose_K`³ 根据 N 生成 K : 先取近似 $(\log_2 N)^2$ 的规模, 再限制在 $[64, 4096]$ 和 $K \leq N$ 的范围内, 最后向下取 2 的幂⁴。Facility 数量为 $\frac{N}{K}$ 。

可逆置换由三轮 Feistel 实现, 对任意键 x , 系统先计算 $g_x = \pi(x)$, 再取其低 $\log N$ 位:

$$g = g_x \text{ and } (N - 1).$$

由于 N 是 2 的幂, 上式等价于 $g_x \bmod N$ 。随后按照

$$r = \lfloor g/K \rfloor, \quad b = g \bmod K.$$

这样获得 Facility 编号 r 和 Router 桶号 b , 如算法 2 所示, 当 N 、 K 、Facility 编号或者 Cubby 容量发生改变时, 既有的 payload 与 MetaEntry 的恢复上下文就会随

-
- 1 Feistel 网络 (将输入分为左右两半、多轮轮函数混合的可逆构造) 的随机种子, 决定可逆置换 $\pi(x)$ 的具体形式。
 - 2 将整数向上取为不小于它的最小 2 的幂, 用作表规模 N 。
 - 3 按 N 计算 Router 桶数 K , 近似为 $(\log_2 N)^2$ 并限制在 $[64, 4096]$ 内, 再向下取 2 的幂且满足 $K \leq N$ 。
 - 4 理论结构要求 K 为多对数规模, 即 $K = O(\text{polylog } N)$ 。当前实现额外要求 K 为 2 的幂, 以便地址拆分与位运算保持一致。

之改变，所以扩缩容迁移不能简单地复制旧槽位的内容。

算法 2 键映射到 Facility 及 Router 桶的流程

- 1: 输入键 x
 - 2: $g_x \leftarrow \pi(x)$
 - 3: $g \leftarrow g_x \text{ and } (N - 1)$
 - 4: $r \leftarrow \lfloor g/K \rfloor$
 - 5: $b \leftarrow g \bmod K$
 - 6: 返回第 r 个 Facility 及其中的 Router $D[b]$
-

3.3 Facility、Cubby 与空闲槽结构

Facility 包括四部分：按 tiered 分组的 Cubbies 集合、tail 及其所有权信息、tiered 上界参数、长度为 K 的 Router 数组。每个 Router 桶维护“键到候选槽位”的局部映射；每个 Facility 在任一时刻只保留一个 tail，这样就能保证系统装填因子能到达 1。

Cubby 包括 tiered(是第几层)、容量、当前元素数、槽位数组、占用槽索引、空闲槽检索结构和变长元数据数组。槽位只保存商压缩后的 payload，不直接保存完整键；完整恢复依赖 MetaEntry、槽位下标与 Facility 上下文。占用槽索引用于迁移扫描和删除回填，避免在空槽上做无效遍历。

FreeSlotTree¹用线性化的线段树去记录 Cubby 内各区间空闲槽数量，初始化的时候，叶层宽度选取不低于 Cubby 容量的最小 2 的幂，合法槽位的叶子计数设为 1。调用 find_free²时，系统从根节点往下找还有空闲计数的子区间，最后返回一个可用槽位；调用 mark_used³或者 mark_free⁴后面就顺着父节点去更新计数，这个结构没有做到理论上的低深度压缩位图效果，而是用比较直观的线段树形式来做守护工作，这样就能在插入，删除以及重建的时候检查空闲状况。

Facility 中存在两种不同的层次概念，需要加以区分，其一为 k-kick⁵，这是在一个 cubby 内部的分层树状管理；其二为 tiered⁶，这是用来管理扩缩容实现的

1 Cubby 内基于线段树维护各区间空闲槽计数的结构，用于快速查找空槽并更新占用状态。

2 在 Cubby 的空闲槽线段树中自根向下查找，返回一个可用槽位下标。

3 将指定槽位标记为已占用，并自底向上更新线段树各区间空闲计数。

4 将指定槽位标记为空闲，并更新线段树计数。

5 Cubby 内部的层级踢出规则，将顺序槽位抽象为 $k + 1$ 层区间并按优先级驱逐。

6 Facility 层面的 Cubby 分级组织，按层级 j 管理不同容量与数量的 Cubby 以支持扩缩容。

分层次 cubbies, 是论文理论的混淆点与核心难点。

3.3.1 tail 管理

tail 是各 Facility 中唯一允许未填满的 Cubby, 在执行插入操作之前, 系统会通过调用 `ensure_tail`¹ 来确保存在一个未满的 tail。如果当前的 tail 已经填满, 则代码会把它移至 `tiers[tail_tier]` 位置, 并且创建一个新的 tier 0 的 tail, 其大致步骤如算法 3 所示:

算法 3 tail Cubby 维护

```
1: Facility F
2: if F.tail 存在且未满 then
3:   返回
4: end if
5: if F.tail_owned 存在且已满 then
6:   将旧 tail 挂入 F.tiers[F.tail_tier]
7:   清空 F.tail
8: end if
9: 创建 1-tiered Cubby, 初始化 slots、meta、free_slots
10: 令 F.tail 指向该 Cubby, 并由 F.tail_owned 持有
```

删除操作时, 如果删除的元素不在 tail Cubby 中, 删除后会从 tail Cubby 中随机取一个元素插入对应的 Cubby 中, 并同步更新 *payload*、*MetaEntry*、*occupied*、*free_slots* 和 *Router*。如果 tail Cubby 中没有元素, 将会获取一个同样 tiered 层次的 cubby, 将其设置为 tail (可能会触发高 tiered 层次 cubby 的 `rebuild_down`)。这样能始终维持非 tail cubby 满载这个不变量。

3.3.2 *j*-tiered cubbies 数量与重建调度

本系统代码实现了 Facility 中多 tiered Cubbies 的基础组织、数量检测和拆分/合并函数。目标数量由 `target_tier_count`²(*j*) 计算:

$$t_j = \left\lfloor \frac{(\log^{(j)} n)^2}{(\log^{(j+1)} n)^2} \right\rfloor,$$

1 保证当前 Facility 存在未满的 tail Cubby; 若当前 tail 已满则将其挂入对应 tier 并新建 tier 0 的 tail。

2 按层级 *j* 与当前元素规模计算该层 Cubby 的目标个数 t_j 。

并至少取 1。Cubby 容量由 $\text{cubby_capacity}^1(K, N, \text{tier})$ 派生：

$$|r_j| \approx \frac{K}{(\log^{(j)} N)^2},$$

同时限制在 $[4, K]$ 范围内。

插入成功后， $\text{maybe_schedule_rebuild}^2$ 检查各 tiered 的 Cubby 数量。若 j -tiered cubbies 的数量大于 $3t_j$ ，调用 rebuild_down^3 ，将 t_j 个 j -tiered cubbies 合并为一个 $(j+1)$ -tiered cubby（数据需要恢复，然后重新依次插入）；若数量小于 $\frac{t_j}{2}$ ，且 $(j+1)$ -tiered cubbies 非空，则调度 rebuild_up^4 ，将其中一个 $(j+1)$ -tiered cubby 拆分为 t_j 个 j -tiered cubbies。

算法 4 Facility tier 重建调度

```

1: for  $j = 0, 1, \dots, F.\text{max\_tier} - 1$  do
2:    $t_j \leftarrow \text{TARGET\_TIER\_COUNT}(j)$ 
3:    $\text{cnt} \leftarrow |F.\text{tiers}[j]|$ 
4:   if  $\text{cnt} > 3t_j$  then
5:     调度  $\text{REBUILD\_DOWN}(F, j)$ 
6:   else if  $t_j \geq 2$  且  $\text{cnt} < t_j/2$  且 tier  $j+1$  非空 then
7:     调度  $\text{REBUILD\_UP}(F, j)$ 
8:   end if
9: end for

```

rebuild_down 会从 j -tiered cubbies 取出 t_j 个 Cubby，扫描其占用槽并恢复键，再创建一个 tier $j+1$ Cubby 重新写入这些键； rebuild_up 则从 tier $j+1$ 取出一个 Cubby，恢复其中所有键，创建 t_j 个 tier j Cubby 并将恢复的原始键全部插入其中。两者都会更新 Router，使 Router 记录指向新的 Cubby 和 Slot。当前实现具备拆分/合并的数据路径，但其调度粒度和容量控制仍属于工程化实现，不能直接推出理论均摊界。

-
- 1 由 K 、 N 与 tier 计算该层 Cubby 的槽位容量，并限制在 $[4, K]$ 内。
 - 2 检查各 tier 的 Cubby 数量是否偏离 t_j ，必要时将合并或拆分任务加入后台重建队列。
 - 3 取出 t_j 个 j 层 Cubby，恢复其中键后合并为一个 $(j+1)$ 层 Cubby 并更新 Router。
 - 4 取出一个 $(j+1)$ 层 Cubby，恢复键后拆成 t_j 个 j 层 Cubby 并重新插入。

3.4 MiniArray 与 MetaEntry

3.4.1 MiniArray 变长元数据存储结构

MiniArray 是 Cubby 组件中用于存储各槽位变长元数据的核心结构，通过多组向量协同维护元数据的长度、偏移、原始内容与连续打包存储区域，实现变长数据的紧凑管理。

MiniArray 各成员变量的功能定义如下：

- `bitlens_[i]`：记录第 i 个元数据条目的二进制位长度；
- `offsets_[i]`：标记第 i 个元数据条目在连续位缓冲区中的起始比特位置；
- `entries_[i]`：保存元数据条目的未压缩原始副本；
- `buf_`：存储所有元数据经重新打包后的连续比特缓冲区。

读取流程

MiniArray 读取流程

```
1: function ACCESS( $i$ )
2:    $bl \leftarrow bitlens\_ [i]$ 
3:    $start \leftarrow offsets\_ [i]$ 
4:   从  $buf\_$  中起始位置  $start$  取出长度为  $bl$  的比特数据
5:   将数据组织为 64 位字数组并返回
6: end function
```

读取操作由 `access`¹ 完成，基于位长度与起始偏移量提取指定元数据并返回标准格式结果：

更新流程

MiniArray 的更新操作由 `update`² 完成，针对指定槽位完成元数据修改，并触发全局重打包以保证存储布局一致性，流程如下：

在理论设计中，MiniArray 可通过内存 B 树与查表法实现均摊常数时间的更新操作。本系统实现保留了变长条目管理、连续比特打包、位数统计的核心接口

-
- 1 按槽位下标从连续缓冲区读出该条变长元数据的比特串。
 - 2 修改指定槽位的元数据并调用 `repack` 重算偏移与缓冲区。

MiniArray 更新流程

```
1: procedure UPDATE(i, bits, bitlen)
2:   if i 越界 then
3:     报错
4:   end if
5:   bitlens_[i]  $\leftarrow$  bitlen
6:   entries_[i]  $\leftarrow$  bits
7:   REPACK
8: end procedure
9: procedure REPACK ▷ 按位长重算偏移并写回 buf_
10:  根据 bitlens_ 重新计算所有条目的偏移量 offsets_
11:  为连续缓冲区 buf_ 分配内存空间
12:  将所有元数据条目按偏移逐位写入 buf_
13: end procedure
```

语义，采用全局重打包策略替代复杂动态结构，在保证功能正确性的前提下简化实现逻辑。

3.4.2 MetaEntry 元数据结构

MetaEntry¹是 Cubby 槽位元数据的结构化表示，用于存储哈希定位、路由校验、插入追踪所需的核心信息，MetaEntry 各字段含义如下：

- **delta**：计算公式为 $g_i - \text{slot_index}$ ，用于根据实际槽位恢复哈希计算得到的 Cubby 偏好槽位；
- **mid_bits**：计算公式为 $g_k / |I|$ ，用于恢复局部桶区间内的中间比特信息；
- **router_bits**：路由校验指纹字段，由 $\text{splitmix64}(\text{key} \oplus \text{gx_pi} \oplus \text{常量})$ 计算后截断为 16 位得到；
- **insert_bits**：插入过程标签，主要用于记录数据插入过程中的 kick 步数信息。

位宽配置与编码规则

MetaCodecLayout²根据哈希表参数 K 与 Cubby 容量动态确定各字段编码位宽：

-
- 1 槽位元数据的结构化字段集合，包含 **delta**、**mid_bits**、**router_bits**、**insert_bits** 等，配合 **payload** 恢复完整置换值 g_x 。
 - 2 由 K 与 Cubby 容量派生 **mid_bits**、**router_bits**、**insert_bits** 等字段的编码位宽。

位宽配置

- 1: $mid_w \leftarrow \text{bit_width}((K - 1)/\text{cubby_capacity})$
 - 2: $rout_w \leftarrow 16$
 - 3: $ins_w \leftarrow 8$
-

MetaEntry 生成逻辑

make_meta_entry^1 是 MetaEntry 结构体的核心生成函数，基于哈希计算结果与插入参数完成字段赋值，核心逻辑如下：

MetaEntry 构造

- | | |
|---|-------------------|
| 1: $g_star \leftarrow g_x \& (N - 1)$ | // 计算全局哈希种子（取模 N） |
| 2: $g_k \leftarrow g_star \& (K - 1)$ | // 提取组内索引（取模 K） |
| 3: $g_i \leftarrow g_k \bmod \text{cubby_capacity}$ | // 计算在 cubby 中的偏移 |
| 4: $\delta \leftarrow g_i - \text{slot_index}$ | // 计算与当前槽位的偏移差 |
| 5: $mid_bits \leftarrow g_k / \text{cubby_capacity}$ | // 提取中间位（组编号） |
| 6: $\text{router_bits} \leftarrow \text{截断}(\text{hash}(\text{key}, g_x), 16)$ | // 路由信息（16 位哈希截断） |
| 7: $\text{insert_bits} \leftarrow \text{ins_tag} \& 0\text{xff}$ | // 插入标签（低 8 位） |
-

函数参数 facility_r 仅作为上下文参数传入，不直接参与字段计算，该参数将在数据恢复阶段与元数据配合完成哈希值的完整重建。

3.5 Router 组件

Router 要维护键到 ($\text{Cubby}^*, \text{slot}$) 的映射关系，其内部条目包含原始键，目标位置以及由 key_bits^2 得到的随机比特串，而节点则保存左右孩子，叶子条目的下标，碰撞桶的下标以及分裂位 split_bit 。

Router 并非按键值大小来组织结构，而是凭借随机比特串形成 Patricia 风格的二叉前缀树³，此前缀树仅在 Router 内部用以识别随机比特串，执行查询时，系统从根节点出发，遵照 split_bit 获取查询键中的随机比特，并决定向左或向右子树探寻；直至抵达叶节点才对比原本的键；要是存在多个键其 64 位随机比特串完全一致，则需执行碰撞桶⁴的线性扫描操作。

-
- 1 根据键、置换值、槽位下标及表参数生成 δ 、 mid_bits 、 router_bits 、 insert_bits 等字段。
 - 2 为键生成 Router 前缀树所用的 64 位随机比特串。
 - 3 Patricia 树是一种压缩前缀树：内部结点仅在有分支处按某一比特位分裂，叶结点存完整键或条目。
 - 4 Patricia 树中随机比特串相同的多个键所落入的线性扫描容器，用于消解哈希碰撞。

算法 5 Router 查询流程

```
1: 输入键  $x$ 
2:  $bits \leftarrow \text{KEY\_BITS}(x)$ ,  $steps \leftarrow 0$ ,  $u \leftarrow \text{root}$ 
3: while  $u$  为合法结点 do
4:    $steps \leftarrow steps + 1$ 
5:   if  $u$  为碰撞桶 then
6:     在桶内逐项比较原始键
7:     返回命中位置或未命中, 以及  $steps$ 
8:   else if  $u$  为叶子 then
9:     比较叶子  $\text{entry}$  中的原始键
10:    返回命中位置或未命中, 以及  $steps$ 
11:   else
12:    按  $bits$  在  $\text{split\_bit}$  处的取值进入左/右子树
13:   end if
14: end while
15: 返回未命中
```

插入或者删除 Router 记录之后, 代码会调用 `rebuild`¹, 并利用 BitWriter 遍历结构来估算 `encoded_bits_`。所以 `bits_total`²可以用来了解路由结构的逻辑编码规模, 但它并不能表示真实的序列化存储或者运行时的内存占用情况, 按照理论中的 Local Query Router 理应可以在紧凑 `bitstream` 上做到常数时间的动态更新; 不过当下的实现则是用整体重建的方式来取代这种复杂的结构。

3.6 商压缩与键恢复

商压缩由槽内 `payload`、MiniArray 中的 `MetaEntry` 以及当前表上下文共同完成。设待处理键为 x , $g_x = \pi(x)$, $\ell = \log N$ 。槽位 `slots[i]` 保存

$$\text{payload} = g_x \gg \ell,$$

即去掉低 ℓ 位后的高位商。低 ℓ 位参与地址拆分和局部恢复, 不直接完整保存。

令

$$g^* = g_x \text{ and } (N - 1), \quad r = \lfloor g^*/K \rfloor, \quad g_k = g^* \text{ and } (K - 1).$$

1 重新计算各 `entry` 的随机比特串, 重建 Patricia 前缀树并更新编码位长统计。

2 返回 Router 前缀树逻辑编码的累计比特数, 用于空间统计。

给定 Cubby 容量 $|I|$ 和实际槽位 i ，代码计算

$$g_i = g_k \bmod |I|, \quad \text{delta} = g_i - i, \quad \text{mid_bits} = \lfloor g_k / |I| \rfloor.$$

恢复时再按

$$g_i = i + \text{delta}, \quad g_k = \text{mid_bits} \times |I| + g_i, \quad g^* = rK + g_k, \quad g_x = (\text{payload} \ll \ell) \mid g^*$$

重建完整置换值。若需要恢复原始键，则继续调用 Feistel 逆置换 π^{-1} 。

rest / payload	Facility 编号 r	中间位 mid_bits	偏好槽位 g_i
$g_x \gg \log N$	$\lfloor g^* / K \rfloor$	$\lfloor g_k / I \rfloor$	$g_k \bmod I $
存入 slots[i]	由地址上下文提供	存入 MetaEntry	由 delta 与槽位恢复

图 3-1 $\pi(x)$ 在商压缩实现中的位段用途

payload 以及 MetaEntry 都依靠表规模，局部桶数，Facility 编号和 Cubby 容量，执行扩缩容迁移之后，元素需先还原成原本的键，然后依照新表的状态加以重新写入，不可直接拷贝旧槽位里的 payload 和 meta。

算法 6 从槽位恢复原始键

- 1: 输入 Facility 编号 r 、Cubby C 、槽位 i
 - 2: **if** i 越界或 $C.\text{slots}[i]$ **then** 为空
 - 3: 返回空
 - 4: **end if**
 - 5: 从 $C.\text{meta}[i]$ 解码 MetaEntry
 - 6: 根据 $r, N, K, C.\text{capacity}, i, \text{payload}$ 重建 g_x
 - 7: **if** 重建失败 **then**
 - 8: 返回空
 - 9: **end if**
 - 10: 返回 $\pi^{-1}(g_x)$
-

3.7 双表渐进迁移

论文提及了两种程度的扩容和缩容，其中一种是多层次的 j-tiered cubbies 设计，这已经实现，另一种是在大规模的扩容缩容，完全是新的表与旧的表进行迁移，例如扩容时，会将旧表中的两个 Facility 的数据插入新表的一个 Facility 中，同理缩容相同的逻辑。当系统哈希表正在进行迁移时，会对应旧表和新表依次执

行对应的插入、删除、查询操作，如果操作完成就提前终止，想要实现这样的无感迁移，就需要实现渐进式的双表迁移¹，具体流程如算法 7 所示。

算法 7 单次迁移预算

```

1: if old_ then 不存在
2:   返回
3: end if
4: 取 migrate_progress_ 指向的旧 Facility
5: 扫描该 Facility 的 tail 和所有 tier Cubby，恢复占用键集合  $S$ 
6: for  $x \in S$  do
7:   按新表参数计算  $g_x = \pi(x)$ ，定位 active Facility
8:   if  $x$  已存在于 active 表 then
9:     跳过
10:  else
11:    调用 ensure_tail 和 kkick_insert2重新写入
12:  end if
13: end for
14: 清空该旧 Facility，推进迁移进度
15: 若所有旧 Facility 均已处理，则释放 old_

```

系统经由 maybe_resize_locked³来判断是否执行扩缩容操作，当下不存在 old 表，而且元素数量超出 $load_factor \times N$ 时，把目标规模设置成 $2N$ 。如果元素数量不为零，又小于 $N/4$ 的话，则把目标规模设定为 $\max\{2, N/2\}$ ，一旦发生这种情况，start_migration⁴就会清空重建调度队列，并把 active 表移至 old_，然后按照新的 N 和 K 创建一个新的 active 表，这个过程中，元素总数 n_+ 保持不变，新增的数据只会被添加到 active 表当中。

迁移由 run_migrate_budget⁵推动，每当插入或者删除操作结束之后，系统最多只会处理一个旧的 Facility。在处理过程中不依靠旧 Router，而是要遍历旧 Facility 的 tail 及其所有的 tier Cubby，经由 recover_key_from_slot⁶（算法 6）来重建原本的键，然后按照新的表参数重新确定位置并执行插入到 active 表的操作，如果某个键已在 active 表中有对应项，则直接忽略这个键，当一个旧 Facility 的处理完毕之后，它的 Router，tiers 和 tail 都会被清除。

-
- 1 当前系统并没有完整地实现渐进式双表迁移，只实现了代码框架，并没有数据支撑；渐进式扩缩容的思想与动态可调整哈希表^[1]等近年方法一脉相承。
 - 2 在持锁状态下根据负载因子判断是否需要扩容或缩容。
 - 3 将当前 active 表移入 old_，按新规模重建 active 表并初始化迁移进度。
 - 4 每次操作后最多迁移一个旧 Facility：恢复其中键并插入 active 表，推进 migrate_progress_。
 - 5 从槽位 payload 与 MetaEntry 重建置换值并求逆，得到原始键。

该迁移方式避免扩缩容时一次性重哈希全部元素。其限制是：查询不会消耗迁移预算；若扩缩容后长期没有插入或删除，old 表会继续保留。此外，由于 payload 和 MetaEntry 依赖新表上下文，迁移必须恢复键并重新插入。

3.8 插入路径与 k-kick 实现

外层函数加锁后调用 `insert_no_lock`¹，成功插入后检查扩缩容，随后执行一个后台重建预算和一个迁移预算。真正写入发生在 `kkick_insert` 中。

插入流程如下：先算出 $g_x = \pi(x)$ ，找到 active 表里对应的 Facility 和 Router 桶；如果 active 表已有此键，则直接返回 `inserted=false`。若是处于迁移窗口，则还要查看 old 表，防止同一个键在迁移过程中被重复写入，确定键不存在之后，系统会调用 `ensure_tail` 来准备好 tail Cubby，并在其中执行 k-kick 操作。

当前 k-kick 是单 Cubby 内的有界踢出链。探测位置由

$$\text{probe_slot}^2(x, |I|, s) = \text{splitmix64}(\pi(x) \oplus s \cdot c) \bmod |I|$$

得到，其中 s 为当前步数， c 为固定常量。若探测槽为空，系统写入 payload 和 MetaEntry，更新 `occupied`、`free_slots` 和 Router；若探测槽已占用，则先恢复被踢出的旧键，删除旧 Router 记录，再用当前键覆盖该槽，并将旧键作为下一轮待插入键。经过 k 轮仍未结束时，系统通过 `FreeSlotTree` 选择空槽兜底。

理论 k-kick 树需要维护更完整的深度、饱和状态和合法驱逐对象。当前实现保留了“单 Cubby 内有限交换”的核心接口，但用伪随机探测链和空槽兜底替代理论结构。因此，实验中的 `kick_count` 表示当前代码中的移动次数，不能直接等同于理论交换成本。

1 在无额外加锁前提下完成插入、扩缩容判断及后台重建/迁移预算。

2 按当前键、Cubby 容量与踢出步数 s 计算本步探针槽位。

算法 8 tail Cubby 内的有界踢出插入

```
1: 输入 Facility  $F$ 、tail Cubby  $C$ 、待插入键  $x$ 
2:  $cur \leftarrow x$ 
3: for  $s = 0, 1, \dots, k$  do
4:    $pos \leftarrow \text{PROBE\_SLOT}(cur, C.capacity, s)$ 
5:   if  $C.slots[pos]$  为空 then
6:     写入  $cur$  的 payload 和 MetaEntry
7:     更新  $occupied$ 、 $free\_slots$  与 Router
8:     返回成功, 移动次数为  $s$ 
9:   end if
10:  从  $pos$  恢复被踢出的键  $kicked$ 
11:  if 恢复失败 then
12:    返回失败
13:  end if
14:  删除  $kicked$  的旧 Router 记录
15:  用  $cur$  覆盖  $pos$ , 并登记新 Router 记录
16:   $cur \leftarrow kicked$ 
17: end for
18: 通过 FreeSlotTree 查找空槽, 写入  $cur$ 
19: 返回成功, 移动次数为  $k + 1$ 
```

3.9 查询与删除路径

查询路径仅读取结构, 不会推进迁移预算, 系统首先在 $active$ 表里定位 Router 桶, 经由 $\text{Router}::\text{locate}^1$ 得到候选位置, 然后用 slot_pi_matches^2 解码 $meta$, 重新创建 $\pi(x)$ 并做比较, 如果 $active$ 表未命中而且 old 表存在, 就对 old 表重复前面的过程, Router 访问步数会被记入 $\text{router_probe_steps}$ 。

删除路径先执行与查询相同的定位和槽位校验。若命中 $active$ 表中的槽位, 系统清空 $slots[slot]$, 将对应 MiniArray 条目更新为空, 移除 $occupied$ 中的槽位下标, 调用 mark_free 标记空闲, 并从 Router 中删除该键。如果被删除的 Cubby 不是 tail, 系统会从 tail 的 $occupied$ 中取出一个元素, 恢复其原始键后搬移到刚刚空出的槽位, 并重新写入 payload、MetaEntry 和 Router 记录。

当处在迁移的状态时, 插入只写入 $active$ 表, 但查询和删除必须同时考虑 $active$ 表和 old 表。删除时若 old 表中仍存在该键, 也需要删除, 否则后续迁移可能把旧键重新搬入 $active$ 表。

1 在 Router 前缀树中查找键, 返回候选 (Cubby, slot) 及访问步数。

2 从候选槽位解码元数据并重建 $\pi(x)$, 与查询键的置换值比对以判定命中。

算法 9 查询命中判定

- 1: 输入查询键 x
 - 2: 在 active 表中定位 (r, b) , 调用 $D[b].locate(x)$
 - 3: **if** Router 返回候选槽位 (C, i) **then**
 - 4: 调用 $slot_pi_matches$ 校验候选槽位
 - 5: 若校验通过, 返回命中
 - 6: **end if**
 - 7: **if** active 表未命中且 old 表存在 **then**
 - 8: 在 old 表中重复定位与槽位校验
 - 9: **end if**
 - 10: 返回最终查询结果
-

第四章 系统验证与性能实验

4.1 实验目的

本章在第三章实现路径的基础上，验证接口语义是否在批量动态操作下保持正确；分析插入、命中查询与删除时间复杂度；分析踢出移动、Router 步数、逻辑元数据位数及扩缩容次数等指标。

4.2 性能实验设置

对每组参数依次执行：批量插入、顺序全命中查询、随机命中查询、删除半数键、混合命中/未命中查询；每阶段统计操作数、吞吐（ops/s）、平均延迟与 p50。命令行参数为 n_insert^1 、 $n_rand_queries^2$ 、 $table_n_hint^3$ 。

为了降低误差的干扰，在 $n \geq 10^4$ 的前提下选取五组参数（删除半数键后 $n = n_{insert}/2$ ），覆盖 $K = 128$ 与 $K = 256$ 两种 Router 桶宽及不同初始 $table_n_hint$ 。

4.3 宏基准结果

表 4-1 汇总五组实验在删除半数键后的表状态 (n, N, K) 及三阶段代表性延迟。查询阶段以随机命中查询为主——更接近线上随机访问；

查询：在 $n \leq 5 \times 10^4$ 时，随机命中查询 p50 稳定在 $2.9\text{--}3.7 \mu\text{s}$ ，吞吐约 $2.5 \times 10^5\text{--}3.2 \times 10^5 \text{ ops/s}$ ，与第三章“Router 定位 + 单槽商压缩校验”的短路径一致。当 $n = 10^5$ 时，p50 升至约 $5 \mu\text{s}$ ，吞吐降至 $1.88 \times 10^5 \text{ ops/s}$ ；Facility 数增至 1024，单次查询的局部索引与缓存局部性成本上升。

插入：各组插入 p50 为 $10\text{--}15 \mu\text{s}$ ，而平均值常为 p50 的 3–5 倍。 $n_{insert} = 3 \times 10^4$

1 批量插入的键数量。

2 随机命中查询的次数。

3 初始化时传给 init 的表规模提示值，用于影响初始 N 的选取。

表 4-1 实验结果（单位：延迟 ns，吞吐 10^3 ops/s）

n_{insert}	hint	n	N	K	插入		随机查询		删半键	
					p50	QPS	p50	QPS	p50	QPS
3×10^4	2^{16}	15000	32768	128	10400	20.7	2900	320.0	5900	39.7
5×10^4	2×10^5	25000	65536	256	11300	27.1	3000	304.4	6400	141.7
5×10^4	2^{20}	25000	65536	256	12900	22.0	3000	301.6	6200	147.4
1×10^5	2^{18}	50000	131072	256	12500	26.0	3700	248.1	7100	131.3
2×10^5	2^{20}	100000	262144	256	14700	23.1	5000	188.3	7800	117.6

一组 $\text{resize_start}^1=11$ ，与加载过程中多次触发扩容一致。

删除：多数配置下删除 p50（6–8 μs ）略高于查询、低于插入，符合“释放槽位 + Router 更新 + 非 tail 回填”的预期。例外是 $n = 15000$ 配置：删半键吞吐仅 3.97×10^4 ops/s，与当次运行中更频繁的 resize 及迁移交错有关，说明删除与扩缩容叠加时会出现阶段性抖动，而非稳态下的常态延迟。

初始规模的影响： $n = 25000$ 、 $N = 65536$ 、 $K = 256$ 的两组实验中， table_n_hint 从 2×10^5 增至 2^{20} 仅改变初始化与插入阶段耗时（插入 QPS 由 27.1k 降至 22.0k，平均延迟由 36.7 μs 升至 45.3 μs ），随机查询 p50 仍约为 3 μs 。这表明在负载因子收敛相近后，查询性能主要由 (N, K) 与 Facility 布局决定，对大 hint 的敏感度低于插入期的表扩张与元数据构建开销。

4.4 结构性指标分析

表 4-2 给出五组实验结束时的累计指标。以下结合第三章机制加以解释。

表 4-2 多规模实验终态累计指标

n	N	K	$\text{moved}_{\text{ins}} / n_{\text{insert}}$	$\text{moved}_{\text{max}}$	$\text{steps}_{\text{max}}$	meta / insert	resize
15000	32768	128	0.94	5	5	8788	11/11
25000	65536	256	0.93	5	5	12677	8/8
25000	65536	256	0.88	5	5	12108	8/8
50000	131072	256	0.94	5	6	14582	9/9
100000	262144	256	0.93	5	5	15387	10/10

踢出次数：五组实验中 $\text{insert_moved_total}^2$ 与插入次数之比稳定在 0.88–0.94， $\text{insert_moved_max}^3=5$ ，说明工程版 k-kick 在单 Cubby 内确有频繁槽位搬移，但单

1 Metrics 中记录的扩容启动次数。

2 全部插入操作累计的 k-kick 搬移次数。

3 单次插入过程中 k-kick 搬移次数的最大值。

次插入触发的链长被常数上界约束,与第三章有界踢出设计一致。`delete_moved_max`¹均为 1, 删除引起的搬移远少于插入。

Router 步数: `router_steps_max`²不超过 6 (仅 $n = 5 \times 10^4$ 、 $N = 131072$ 时出现 6), 表明在当前随机种子与规模下 Patricia 路由深度未失控; `router_steps_total`³随总操作数近似线性增长, 平均每步操作约 1–2 次路由访问, 与“先 Router 再单槽校验”的路径相符。

逻辑元数据: `avg_meta_bits_per_insert`⁴从约 8.8×10^3 ($K = 128$) 增至约 1.5×10^4 ($n = 10^5$), `meta_bits_max`⁵同步上升。该量为 MiniArray/Router 编码的逻辑位数累计, 反映结构维护随规模加重; 但其随规模单调上升的趋势表明, 元数据维护是插入期除踢出外的另一主要成本, 且对 K 与 Facility 数量敏感。

扩缩容: `rebuild_up`⁶/`down`⁷均为 0, 规模变化主要由 active/old 双表渐进迁移完成; `resize_start` 与 `resize_finish`⁸相等, 表示每次扩容均完成收尾。 $n = 15000$ 终态对应 11 次 `resize`, 高于其余组 (8–10 次), 与表 4-1 中该组插入/删除尾部延迟偏大相互印证。

4.5 小结

在 $n \geq 10^4$ 的负载下: (1) 命中查询保持亚 $10 \mu s$ 的 p50, 直至 $n = 10^5$ 时因 Facility 增多而适度上升, 未观察到路径长度失控; (2) 插入平均延迟显著高于 p50, 且与踢出、元数据及 `resize` 次数相关; (3) 累计指标显示踢出链长有界、Router 深度有界、逻辑元数据随规模单调增加。上述结果支持第三章实现可在中等规模动态负载下稳定运行, 同时也暴露删除与扩缩容交错时的延迟抖动, 以及大规模下查询吞吐的温和下降——二者可作为后续优化调度与局部更新的实证依据。

1 单次删除过程中元素搬移次数的最大值。

2 单次操作中 Router 访问步数的最大值。

3 全部操作累计的 Router 访问步数。

4 每次插入平均累计的 MiniArray/Router 逻辑编码比特数。

5 单次操作写入元数据逻辑比特数的峰值。

6 Facility 层 tier 向上合并 (拆分高层 Cubby) 的执行次数。

7 Facility 层 tier 向下合并 (合并低层 Cubby) 的执行次数。

8 双表扩缩容完成次数; 与 `start` 相等表示迁移均已收尾。

第五章 总结与展望

5.1 工作总结

本文就 Bender 等人^[2]在《On the Optimal Time/Space Tradeoff for Hash Tables》中提出的主要结构展开论述，并精简设计出一个哈希系统，该系统把 HashTable 设作入口，内部按照 Facility, Cubby, Slot 这三层来组织元素；设计一个可逆的哈希函数；利用 Router 来维持局部键与槽位之间的映射关系；在槽位处存储经过商压缩之后的 payload；凭借 MiniArray 来保留用于恢复 $\pi(x)$ 的 MetaEntry；在执行扩缩容操作的时候则采取 active/old 双表逐步过渡的方式。

功能检测覆盖初始化、插入、查询、删除、删后查询与重复插入等路径。性能检测在 $n \geq 10^4$ 的多组 (n, N, K) 配置下开展：命中查询 p50 在 2.9–5 μs 量级，插入 p50 约 10–15 μs ，insert_moved_max 与 router_steps_max 均不超过 6，与第三章有界路由及工程版 k-kick 的设计一致。

5.2 不足与局限

本文存在一些不足与局限之处，与论文中的完整理论结构相比仍有所差距，主要体现在以下方面：

1. k-kick 机制：当前实现已在单个 Cubby 内达成有界踢出链，但尚未完整模拟论文中的 k-kick 树层次结构；实验中的 kick_count 反映的是代码层面的搬移次数，不能直接等同于理论交换成本。
2. Facility 层重建：虽具备多 tiered Cubby、tail 维护及拆分/合并接口，但调度器每次执行完整重建，未将 $O(|I|)$ 级任务细分为严格的后台步骤；tiered 上限与容量参数仍属工程设定，尚不足以支撑理论均摊界的论证。
3. MiniArray 与 Router 更新：两者保留了变长编码与局部路由的接口语义，但更新分别依赖全局重打包与整树重建，未达到论文所要求的常数时间局部

更新。

4. 空间统计: `meta_bits_total`、Router 编码位数等指标仅反映逻辑编码规模, 未计入 `vector` 指针、对象头等运行时内存开销, 不能作为严格空间上界的证明依据。

实验范围仍存在局限: 虽已在 $n \in [1.5 \times 10^4, 10^5]$ 上扫过多组 (N, K) , 但未变化负载因子与随机种子分布, 也未与标准库或其它开源哈希表对比, 结论仅适用于当前实现策略下的行为刻画, 不能外推为通用哈希表的最优性能。

5.3 可改进方向

针对上述不足, 后续工作可从以下方向推进:

1. 在 Cubby 内完整实现 k-kick 树结构, 使其与论文中的层级控制及失败处理机制相衔接。
2. 细化 Facility 层后台重建调度, 将拆分/合并拆为可分步执行的任务, 并重新校准 tiered 上限、 t_j 与 Cubby 容量公式。
3. 改进 MiniArray 与 Router 的增量更新策略, 降低重打包与整树重建带来的开销, 并区分逻辑编码位数与实际内存占用。
4. 扩展测试覆盖面: 增加负载因子、随机种子等控制变量, 补充与标准实现的对比实验及可视化分析, 更充分地刻画系统的时间与空间行为。

参考文献

- [1] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. Modern Hashing Made Simple[C/OL]//Proceedings of the 2024 SIAM Symposium on Simplicity in Algorithms (SOSA 2024). SIAM, 2024: 363-373. DOI: 10.1137/1.9781611977936.33.
- [2] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[C/OL]//Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC '22). New York, NY, USA: ACM, 2022: 1284-1297. DOI: 10.1145/3519935.3519969.
- [3] KUSZMAUL W, WALZER S. Space Lower Bounds for Dynamic Filters and Value-Dynamic Retrieval[C/OL]//Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC 2024). ACM, 2024: 1153-1164. DOI: 10.1145/3618260.3649649.
- [4] BENDER M A, CONWAY A, FARACH-COLTON M, et al. Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once[J/OL]. Journal of the ACM, 2024, 71(1): 3:1-3:49. DOI: 10.1145/3625817.
- [5] LEHMANN H P, SANDERS P, WALZER S. SicHash – Small Irregular Cuckoo Tables for Perfect Hashing[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023). SIAM, 2023: 176-189. DOI: 10.1137/1.9781611977561.ch15.
- [6] LEHMANN H P, MUELLER T, PAGH R, et al. Modern Minimal Perfect Hashing: A Survey[J/OL]. ACM Computing Surveys, 2026, 58(10): 251:1-251:36. DOI: 10.1145/3797036.
- [7] PANDEY P, BENDER M A, CONWAY A, et al. IcebergHT: High Performance Hash Tables Through Stability and Low Associativity[J/OL]. Proceedings of the

- ACM on Management of Data, 2023, 1(1): 47:1-47:26. DOI: 10.1145/3588727.
- [8] AAMAND A, KNUDSEN J B T, THORUP M. Load Balancing with Dynamic Set of Balls and Bins[C/OL]//Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021). ACM, 2021: 1262-1275. DOI: 10.1145/3406325.3451107.
- [9] BANSAL N, KUSZMAUL W. Balanced Allocations: The Heavily Loaded Case with Deletions[C/OL]//Proceedings of the 63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS 2022). IEEE, 2022: 801-812. DOI: 10.1109/FOCS54457.2022.00081.

致 谢

行文至此，落笔为终。感谢遇到的导师、感谢身边的每一个同学、感谢遇到的每一个人。凡是过往，皆为序章，很喜欢看到的一句话，人生如棋，落子无悔。愿我们跃入人海，各自灿烂。